
**Large-Scale Design & Analysis of Neural
Networks
System Requirements Specification
Version 2.5
2/4/2025**

Document Control

Distribution List

The following list of people will receive a copy of this document every time a new version of this document becomes available:

Teaching assistants:

Clay Pate

Customer(s): M. Ilhan Akbas

John M. Thompson

Project team members:

Daniel Machado

Simon Cole

Muhammed Sharif Adeptu

Leah Powell

Change Summary

The following table details changes made between versions of this document:

Version	Date	Modifier	Description
0.1	10/15/2024	Leah Powell	Document Creation
1.0	10/29/2024	Leah Powell	Sections 1-7
1.1	11/14/2024	Leah Powell	Sections 1.4, 7.2
1.2	11/20/2024	Muhammed-Sharif	Edits to sections 1.6, 2.2, 2.4
1.3	11/25/2024	Muhammed-Sharif	Sections. 2.5, 2.6, 2.7
1.4	11/26/2024	Leah Powell	Sections 3-6
1.5	11/26	Simon Cole	Added behavioral requirements
1.6	11/29	Daniel Machado	Section 3.1 User Interface Requirements
1.7	11/29	Daniel Machado	Added DFD level 0 1 and 2
2.0	12/1/2024	Leah Powell	3.3, 6.1, 6.2, 7.3
2.1	12/5/2024	Leah Powell	4.3
2.2	1/28/2025	Leah Powell	6.1, 6.2, 7.1, 1.4, 2.3, 8, multiple formatting issues
2.2	1/30/2025	Simon Cole	Environmental impact, 1.4
2.3	2/2/2025	Daniel Machado	Section 8.2 safety
2.4	2/3/2025	Muhammed-Sharif	Section 8.3 Welfare, 8.4 Social
2.5	2/4/2025	Leah Powell	Section 8.1 Health, 8.4 Economic, 1.4, 2-6

	2/4/2025	Simon Cole	Environmental impact, safety
--	----------	------------	------------------------------

Table of Contents

Document Control	ii
Distribution List.....	ii
Change Summary	ii
1. Introduction	1
1.1. Purpose and Scope	1
1.2. Intended Audience and Reading Suggestions	1
1.3. Document Conventions	1
1.4. Project References	1
1.5. Definitions, Acronyms, and Abbreviations	3
1.5.1. Definitions	3
1.5.2. Acronyms.....	3
1.5.3. Abbreviations	4
2. General Description	5
2.1. Product Perspective.....	5
2.2. Product Features	5
2.3. User Classes and Characteristics	5
2.3.1. Actors	6
2.3.2. Use Cases	7
2.3.3. Scenarios.....	7
2.4. General Constraints.....	9
2.5. Operating Environment.....	9
2.6. User Documentation	10
2.7. Assumptions and Dependencies	10
3. External Interface Requirements.....	11
3.1. User Interfaces	11
3.2. Hardware Interfaces	12
3.3. Software Interfaces.....	12
3.4. Communications Interfaces	12
4. Behavioral Requirements.....	13
4.1. Same Class of User.....	13
4.2. Related Real-World Objects	13
4.3. Stimulus.....	13

4.4.	Related Features	14
4.5.	Functional.....	14
5.	Non-behavioral Requirements.....	15
5.1.	Performance Requirements	15
5.2.	Safety Requirements	15
5.3.	Qualitative Requirements	15
5.3.1.	Availability	15
5.3.2.	Security.....	15
5.3.3.	Maintainability	15
5.3.4.	Portability	15
5.4.	Design and Implementation Constraints	16
6.	Other Requirements.....	17
6.1.	Database Requirements	17
6.2.	Operations.....	17
7.	Analysis Models	18
7.1.	<i>Data Flow Model</i>	18
7.1.1.	<i>Data Sources</i>	18
7.1.2.	<i>Data Sinks</i>	18
7.1.3.	<i>Data Dictionary</i>	18
7.1.4.	<i>Context Diagram (Level 0 Data Flow Diagram)</i>	19
7.1.5.	<i>Level 1 Data Flow Diagram</i>	19
7.1.6.	<i>Level 2 Data Flow Diagram</i>	20
7.2.	<i>Class Model</i>	21
7.3.	<i>State Model</i>	23
8.	Engineering Solutions in Real-World	24

1. Introduction

1.1. Purpose and Scope

This SRS covers the scope of the Large-Scale Design & Analysis of Neural Networks senior design project which aims to streamline the process of implementing deep learning using Bayesian Neural Networks. This is to address the lack of systemic optimization of a Neural Network's architecture from a statistical standpoint using the gradient of cost functions. This includes creating visualization tools for determining loss criteria and using histograms to analyze the weight distributions and convergence success for the neural networks created in this pipeline process.

The application of this system would be beneficial to the goals of any field in neural networks science. This is especially regarding the long-term accuracy of the training that results from these neural networks and the much-needed organization required to make these pipelined neural networks more efficient and consistent in accuracy/convergence.

1.2. Intended Audience and Reading Suggestions

This SRS document is intended to be used by any stakeholders of the project, members of the development team, or other collaborative parties such as teaching assistants. The rest of the SRS contains a product descriptive section (2.0) followed by a series of types of requirements. These include behavioral requirements, non-behavioral requirements, and any other non-specified requirements that might be necessary such as database requirements. After that are the project models which were specifically selected as appropriate for this project type and design. A typical reading sequence would be to start with the Introduction and Description, then read through the Analysis Models, then finally the requirements sections as directed.

1.3. Document Conventions

This document follows the typical template conventions.

1.4. Project References

[1] S. Adepetu, S. Cole, D. Machado, and L. Powell, *Project Proposal Document*. 2024.

[2] S. Adepetu, S. Cole, D. Machado, and L. Powell, *Software Design Documentation*. 2024.

- [3] V. Fortuin, "Priors in Bayesian Deep Learning: A Review," *International Statistical Review*, May 2022, doi: <https://doi.org/10.1111/insr.12502>.
- [4] J. Li, X. Yuan, and Ercan Engin Kuruoglu, "Exploring Weight Distributions and Dependence in Neural Networks with α -Stable Distributions," *IEEE Transactions on Artificial Intelligence*, pp. 1–11, Jan. 2024, doi: <https://doi.org/10.1109/tai.2024.3409673>.
- [5] Kaplan, Jared, et al. "Scaling laws for neural language models." arXiv preprint arXiv:2001.08361 (2020).
- [6] Ludvigsen, Kasper Groes Albin. "How to Estimate and Reduce the Carbon Footprint of Machine Learning Models." Towards Data Science, 23 Jan. 2025, towardsdatascience.com/how-to-estimate-and-reduce-the-carbon-footprint-of-machine-learning-models-49f24510880/.
- [7] R. Miotto, F. Wang, S. Wang, X. Jiang, and J. T. Dudley, "Deep Learning for healthcare: review, Opportunities and Challenges," *Briefings in Bioinformatics*, vol. 19, no. 6, pp. 1236–1246, 2018, doi: <https://doi.org/10.1093/bib/bbx044>
- [8] A. Pollen, "Deep Learning In Healthcare | Future Of Healthcare," Healthcare AI | Aidoc Always-on AI, Feb. 17, 2020. <https://www.aidoc.com/learn/blog/deep-learning-in-healthcare/>
- [9] "Artificial Intelligence Risks: Black-Box Reasoning | MedPro Group," [www.medpro.com](https://www.medpro.com/artificial-intelligence-risks-blackboxreasoning). <https://www.medpro.com/artificial-intelligence-risks-blackboxreasoning>
- [10] <https://kodzilla.pl> and dev@kodzilla.pl, "Addepto," Addepto, Oct. 18, 2019. <https://addepto.com/blog/machine-learning-in-economics-how-is-it-used/>
- [11] Y. Shen and X. Zhang, "The impact of artificial intelligence on employment: the role of virtual agglomeration," *Humanities and Social Sciences Communications*, vol. 11, no. 1, pp. 1–14, Jan. 2024, doi: <https://doi.org/10.1057/s41599-024-02647-9>

- [12] Sverdlik, Anna, et al. “The PhD Experience: A Review of the Factors Influencing Doctoral Students’ Completion, Achievement, and Well-Being.” *International Journal of Doctoral Studies*, vol. 13, 2018, pp. 361–388, <https://doi.org/10.28945/4113>.
- [13] Amodei, Dario. “Machines of Loving Grace: How AI Could Transform the World for the Better.” Anthropic, Oct. 2024, <https://darioamodei.com/machines-of-loving-grace#taking-stock>

1.5. Definitions, Acronyms, and Abbreviations

1.5.1. Definitions

This section lists terms used in this document and their associated definitions.

Table 1: Term Definitions

Term	Definition
Pytorch	A machine-learning library based on the open-source library on Lua.
Baye’s Theorem	A theorem on probability, or statistical inference, that prior probability can be combined with new values and evidence to create a posterior probability which is updated as it becomes available.
Bayesian Neural Networks	Neural Networks whose weight distributions are modeled after probabilities using Baye’s Theorem.
Architecture	Structural and logical layout and overall organization of a Neural Network.
Bins	Grouping continuous data into categories or intervals to use data as if it were discrete.
Weights	The connections between nodes/units in a neural network.
Histogram	Visualizing the distribution of continuous data into a graph and grouping into bins, similar to a bar-graph.
Convergence	Refers to the point where a model/network [learning] reaches a stable state and changes to its performance are at a minimum.
Tensors	Multidimensional arrays to store data used by neural networks

1.5.2. Acronyms

This section lists the acronyms used in this document and their associated definitions.

Table 2: Acronym Definitions

Term	Definition
SDD	System Design Document
NN	Neural Network
BNN	Bayesian Neural Network

BN	Batch Normalization
DL	Deep Learning
SRS	Software Requirements Specification
ML	Machine Learning
CONV	Convergence

1.5.3. Abbreviations

This section lists the abbreviations used in this document and their associated definitions.

Table 3: Abbreviations Definitions

Term	Definition
e.g.	For example
Etc.	Ex cetera

2. General Description

2.1. Product Perspective

This product serves to improve the structural organization and implementation of designing Neural Networks, or Deep Learning Systems. Results from this product will affect development in areas such as:

Network generation: Providing tools for automated creation of diverse network architectures based on user-defined parameters.

Networks initialization: Incorporating probabilistic methods to improve weight initialization and enhance training outcomes.

Pipeline optimization: Offering a structured framework to evaluate and improve network convergence and accuracy over time.

The core objective is to streamline the development process, making it easier for researchers and developers to build high-performing models efficiently.

2.2. Product Features

To achieve the goals outlined above, this product offers a suite of tools that empower users to analyze and optimize neural networks effectively. Key features include:

- **Network Loader:** A tool for quickly and efficiently loading a large set of neural networks into a project environment. This eliminates the manual effort of managing multiple architectures and datasets, enabling users to scale their experiments easily.
- **Loss Analysis Tool:** A visualization tool for analyzing loss curves to determine whether a network is converging effectively. This feature provides users with real-time feedback on their training results, helping them identify issues early in the process.
- **Weight Binning and Visualization:** A feature that bins network weights into discrete ranges and visualizes their distributions through histograms. This helps users understand the behavior of their network weights during training and assess whether the weights are stabilizing over time.
- **Success Criteria Evaluation:** A customizable tool that allows users to define success criteria for their networks, such as target accuracy, loss thresholds, or convergence speed. This ensures the generated networks meet user-defined performance benchmarks.
- **Export and Reporting:** Users can export training results, including loss graphs and weight distributions, in a standardized format for further analysis or sharing with collaborators.

2.3. User Classes and Characteristics

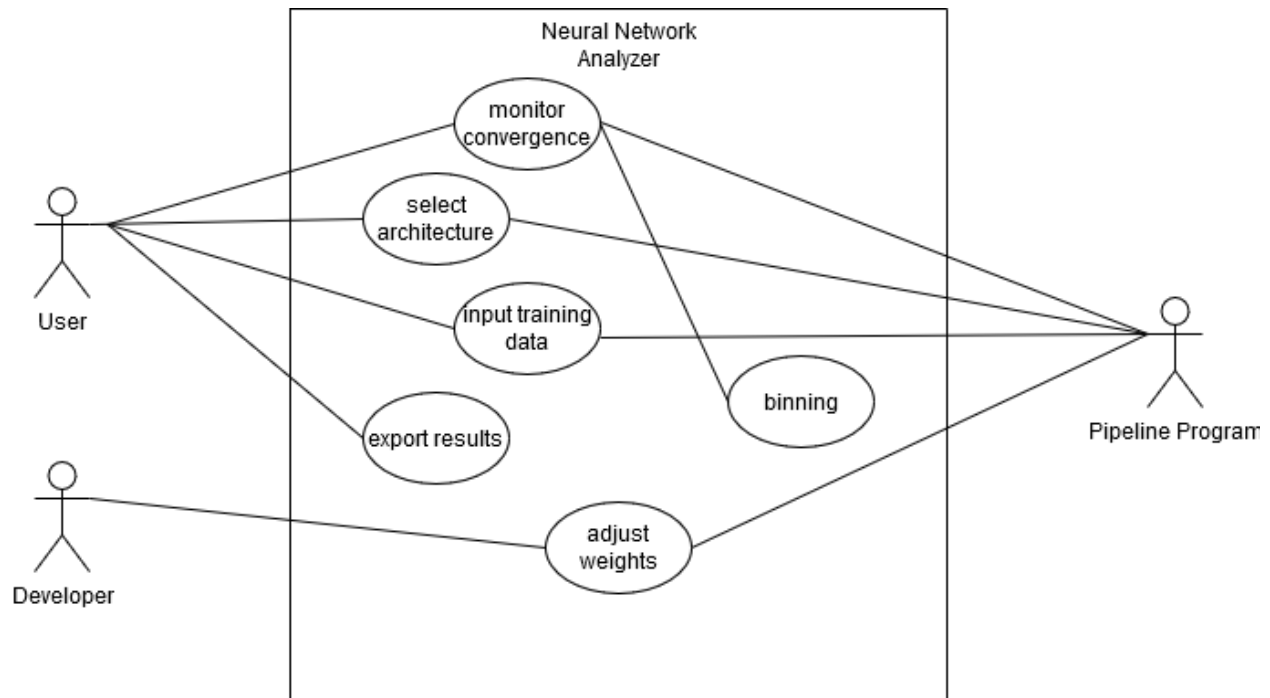


Figure 1. Use Case diagram of the system

2.3.1. Actors

This section presents the actors in the system.

- User:
 - A user runs the program with the intention of using its main features without any writing or maintenance to the program.
- Developer:
 - A developer is the user which initially writes and later maintains the system and adjusts the weights within the system.
- Pipeline Program:
 - The program is an automated component of the system that manages the workflow of creating, training, and analyzing neural networks.

2.3.2. Use Cases

This section presents the Use Cases, developed for the system.

- **Input Training Data:** Provide datasets for training.
- **Monitor Convergence:** Track if networks meet defined success criteria.
- **Select Architecture:** Choose or define network structures.
- **Export Results:** Save analysis and training outcomes.
- **Adjust Weights:** Modify network weight parameters.
- **Binning:** Group and analyze network weights into ranges

2.3.3. Scenarios

This section presents scenarios for **each use case**, described in the previous section.

Scenario 1: Input Training Data

Description: User inputs files to be used by the system to train the model

Actors: User, Pipeline Program

Precondition: The user has a .csv files prepared in program

Trigger Condition: User launches the program

Steps:

1. The system reads the config file to retrieve the file paths and parameters
2. The system ingests the input data from the .csv file in the data directory
3. The program parses the .csv file, extracting input (a, b_ and output (y) column.
4. The data is converted into PyTorch tensors for training.
5. The pipeline confirms process is complete & ready for the next stage.

Scenario 2: Monitor Convergence

Description: The user wishes to track the training progress of the networks.

Actors: User, Pipeline Program

Precondition: Network training has already begun and a convergence criterion is defined.

Trigger Condition: User initiates NetworkAnalyzer.ipynb

Steps:

1. The system initializes the tracking of loss per epoch.
2. The system checks if loss improvement is below convergence threshold.
3. The system determines for training to continue or stop.
4. The system displays the status in convergence.

Scenario 3: Select Architecture

Description: The user chooses a model architecture for training, between different predefined architectures.

Actors: User, Pipeline Program

Precondition: The system supports multiple predefined architectures, and user has not yet started training.

Trigger Condition:

Steps:

1. The user selects either SimpleNet, LargerNet, or another available architecture.
2. The system initializes the selected model.
3. The system confirms the architecture selection.
4. The system begins the neural network training.

Scenario 4: Export Results

Description: The user wishes to export saved neural network results, including performance metrics and processed histogram data.

Actors: User, Pipeline Program

Precondition: Neural networks have completed training, with working and broken networks saved into their proper directories.

Trigger Condition: User initiates NetworkAnalyzer.ipynb

Steps:

1. The system looks for trained neural networks from stored model's directory.
2. The system finds & loads those stored models.
3. The system extracts weight values from layers, and flattens those into bins.
4. The system generates and displays the weight distribution histogram plots.
5. The system fits gaussian curves to weight distributions.
6. The system saves the histogram data.
7. The system automatically saves results after training and analysis

ALT

- 4.2. The system does not find those stored model's directory.
- 4.3. The system creates the directory upon it not found existent.
- 4.4. The system follows the rest of the steps as normal.

Scenario 5: Adjust Weights

Description: The developer wishes to modify network weights in a structured way manually before training.

Actors: Developer, Pipeline Program

Precondition: The network model has been instantiated and the system can access its tensors, and training has not begun yet.

Trigger Condition: The developer initiates NetworkAnalyzer.ipynb

Steps:

1. The system initializes a model architecture with its default weights.
2. The system checks if weight masks are to be applied, which are hard coded into the system.
3. The system initializes weights using the mask.
4. The system begins training with the modified weights.

ALT

- 3.1 The system finds no mask is to be applied.
- 3.2 The system training proceeds with default weight initialization.

2.4. General Constraints

PyTorch Library Dependency:

The system must rely on the PyTorch library, which requires compatibility with Python 3.8 or newer and installation of all necessary PyTorch dependencies.

Operating System Dependency:

The system must be compatible with common operating systems like Windows 10, Ubuntu 20.04, or macOS Ventura to support a broad user base.

Resource Allocation:

When training networks on the GPU, training tasks must optimize GPU resource usage to prevent bottlenecks, such as limiting batch sizes to avoid exceeding GPU memory.

2.5. Operating Environment

The software for the project that controls network creation runs in Jupyter Notebook and was tested using a macOS M2 chip to achieve the specific results presented. Similar results can be achieved on other systems with sufficient processing power and GPU capabilities.

2.6. User Documentation

User documentation components are currently not required or implemented.

2.7. Assumptions and Dependencies

Assumptions

1. Users have access to a compatible Python environment (Python 3.8 or newer) and can install the required libraries, including PyTorch.
2. Users are able to install the package through PIP
3. Users are familiar with Jupyter Notebook as the primary interface for utilizing the software.
4. The system assumes that users have basic knowledge of neural networks and their training processes.
5. The input training data provided by the user is clean, properly formatted, and free of significant errors or missing values.
6. Users have access to sufficient computational resources, such as a modern CPU or GPU, for running the training tasks efficiently.

Dependencies

1. The software is dependent on the PyTorch library for implementing and training neural networks.
2. CUDA is required for leveraging GPU acceleration during training on systems with NVIDIA GPUs.
3. The system depends on standard Python libraries, such as NumPy and Matplotlib, for data processing and visualization.
4. Internet access is required for installing software dependencies and installing the package through PIP.

3. External Interface Requirements

3.1. User Interfaces

The following are the requirements related to the initialization of the system.

[REQ-UI1] The system shall allow the user to instantiate the network with a predefined architecture.

[REQ-UI2] The system shall allow the user to input the convergence threshold.

[REQ-UI3] The system shall allow the user to customize the success loss to evaluate each model.

[REQ-UI4] The system shall allow the user to input the number of networks to produce.

[REQ-UI5] The system shall allow the user to input the max number of networks. If the user does not input the maximum number of networks, it shall default to 1.3 times the number of networks to produce.

The following are requirements related to the training of the model.

[REQ-UI7] The system shall allow the user to input the test data loader.

[REQ-UI8] The system shall allow the user to input the training data loader.

[REQ-UI9] The system shall allow the user to input the number of epochs to train the networks for.

[REQ-UI10] The system shall allow the user to input the loss function to use.

[REQ-UI11] The system shall allow the user to input the learning rate to be used for training.

[REQ-UI12] The system shall allow the user to display the loss graph of a single trained network. This is to be done before starting the training of the many networks.

[REQ-UI13] The system shall allow the user to display the histogram for the weight distribution of a specific layer.

[REQ-UI14] The system shall allow the user to step through the weights.

[REQ-UI15] The system shall accept weight distributions as input in the form of a 4D array with dimensions: (layer, neuron, incoming weight, bin).

[REQ-UI16] The system shall allow the user to specify the traversal strategy using the parameter `walkthrough_strategy`.

[REQ-UI17] The system shall allow the user to select a specific layer for analysis using the parameter `layer`.

[REQ-UI18] The system shall accept `bin_edges` as input, representing the edges of the bins for the selected layer to label the x-axis with actual values.

[REQ-UI19] The system shall allow the user to optionally specify the starting neuron for the walkthrough using the parameter `start_neuron`.

[REQ-UI20] The layer index shall be 0-indexed.

3.2. Hardware Interfaces

Not applicable to this project.

3.3. Software Interfaces

[REQ-SI1] The system shall run with files provided by the user for architecture and training data.

[REQ-SI2] The system shall interface with the operating system to validate and retrieve file paths for configuration files, data files, and model output directories.

[REQ-SI3] The system shall integrate with PyTorch for neural network training and inference.

[REQ-SI4] The system shall output trained model state dictionaries using PyTorch.

3.4. Communications Interfaces

[REQ-CI1] The system shall be run using network connection.

4. Behavioral Requirements

4.1. Same Class of User

[REQ-SCU1] The system shall have only one level of access privilege to use across developers and users of the system.

4.2. Related Real-World Objects

Not applicable to this project.

4.3. Stimulus

[REQ-STIM1] The system shall automatically generate k Neural Networks based on architecture and training data provided by the user.

[REQ-STIM1.2] The system shall collect a number input from the user to generate a defined number of NNs.

[REQ-STIM1.3] The system shall train the generated networks with training data provided by the user.

[REQ-STIM1.4] The networks generated in requirement STIM1 shall be implemented in some user-defined NN architecture.

[REQ-STIM2] The system shall evaluate each neural network based on user-defined success criteria.

[REQ-STIM2.2] The system shall only save NNs, which meet a user defined loss-threshold.

[REQ-STIM2.3] The system shall save NNs which did not meet this threshold into a separate file.

[REQ-STIM2.4] The system shall display to the user the number of NNs successfully trained and the number failed.

[REQ-STIM3] The user shall be able to specify different training hyperparameters for each neural network generated, such as learning rate or batch size.

[REQ-STIM3.2] The system shall take in the NN learning rate as input.

[REQ-STIM3.3] The system shall take in the loss function to be used as input.

[REQ-STIM3.4] The system shall take in the loss threshold to define which networks succeed in training as input.

[REQ-STIM3.5] The system shall take in the maximum allowable difference in loss between the last two epochs as input.

4.4. Related Features

[REQ-RF1] The system shall have a tool which generates discretized bins for each weight of some NN.

[REQ-RF2] The system shall select each weight in the architecture of a set of NNs generated by the system.

[REQ-RF3] The system shall find the minimum and maximum weights in each layer of the network architecture.

[REQ-RF4] The system shall partition weights into equal width bins between the minimum weight value of the layer and the maximum.

[REQ-RF5] The number of bins to partition into shall be provided by the user.

[REQ-RF6] The system shall then iterate through every weight in all the networks and accumulate its corresponding bin.

[REQ-RF7] The system shall populate bins with the number of occurrences of that weight, in that range, across all NN's.

4.5. Functional

[REQ-F1] the system should generate histograms according to the cumulation of weights.

[REQ-F2] The system shall alert the user if the maximum iterations is reached without success criteria being met.

[REQ-F3] The system shall allow the user to specify a maximum number of training epochs.

[REQ-F4] The system shall allow the user to specify a maximum number of attempts to generate a successful model.

[REQ-F5] The system shall allow the user to specify the loss criterion from the pytorch class.

[REQ-F6] The system shall be able to generate a graph displaying the loss of the neural network, calculated using the specified loss criterion.

[REQ-F7] The system shall be able to identify unique distributions.

5. Non-behavioral Requirements

5.1. Performance Requirements

[REQ-PR1] The system shall have a tool for systemic generation of loss graph.

[REQ-PR2] The system shall display the generated loss graph(s).

[REQ-PR3] The system shall use the loss graph tool to define the success criteria.

5.2. Safety Requirements

Not applicable to this project.

5.3. Qualitative Requirements

[REQ-QR1] The system shall be capable of handling increased data volumes without significant degradation in performance.

[REQ-QR2] The system shall have consistent and reproducible results from neural network training.

5.3.1. Availability

[REQ-Av1] The system shall allow the user to re-run the model if the network fails.

5.3.2. Security

[REQ-S1] The system shall log and save generated network data in a specified file location.

5.3.3. Maintainability

[REQ-M1] The system shall be structured with readable and well-commented code without grammar mistakes.

5.3.4. Portability

[REQ-P1] The system shall be able to run after installation of package from

5.4. Design and Implementation Constraints

[REQ-DIC1] The software shall be developed in Jupyter Notebook using Pytorch.

[REQ-DIC2] The software shall be uploaded to PyPI and be installable by anybody using “pip install nnanalyzer”

6. Other Requirements

6.1. Database Requirements

[REQ-DR1] The system shall store training data entities, including input features and target labels in a structured format that supports PyTorch tensor conversion.

[REQ-DR2] The system shall maintain data about each training session, including model configurations, hyperparameters, and loss metrics.

[REQ-DR3] The system shall save successful networks.

6.2. Operations

[REQ-O1] The user shall be able to save output network data.

[REQ-O2] The user shall be able to keep networks that meet success criteria and disregard those that do not meet it.

[REQ-O1] The system shall allow users to initiate model training with specified hyperparameters.

[REQ-O2] The system shall have operations to analyze data and organize into histograms for weights.

[REQ-O4] The system shall allow unattended operations while automated training is occurring and the saving of models without requiring user interaction during runtime.

7. Analysis Models

7.1. Data Flow Model

7.1.1. Data Sources

The data sources and their inputs to the system identified in the data flow model are as follows:

- Network Layers
- Binning mechanism
- Histogram
- Pipeline

7.1.2. Data Sinks

The data sinks and their system outputs identified in the data flow model are as follows:

- Network Layers
- Tools
- Network Program

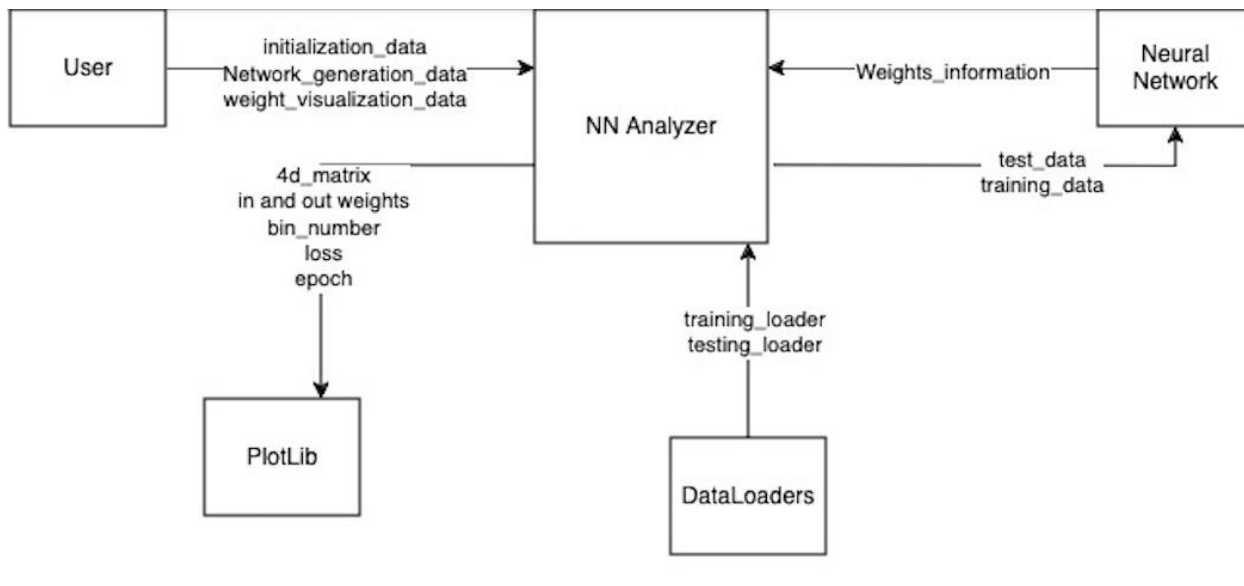
7.1.3. Data Dictionary

The data types are described in the data dictionary below. This section includes the name of the data type; a description of the contained data; how the data is structured; and the range of values.

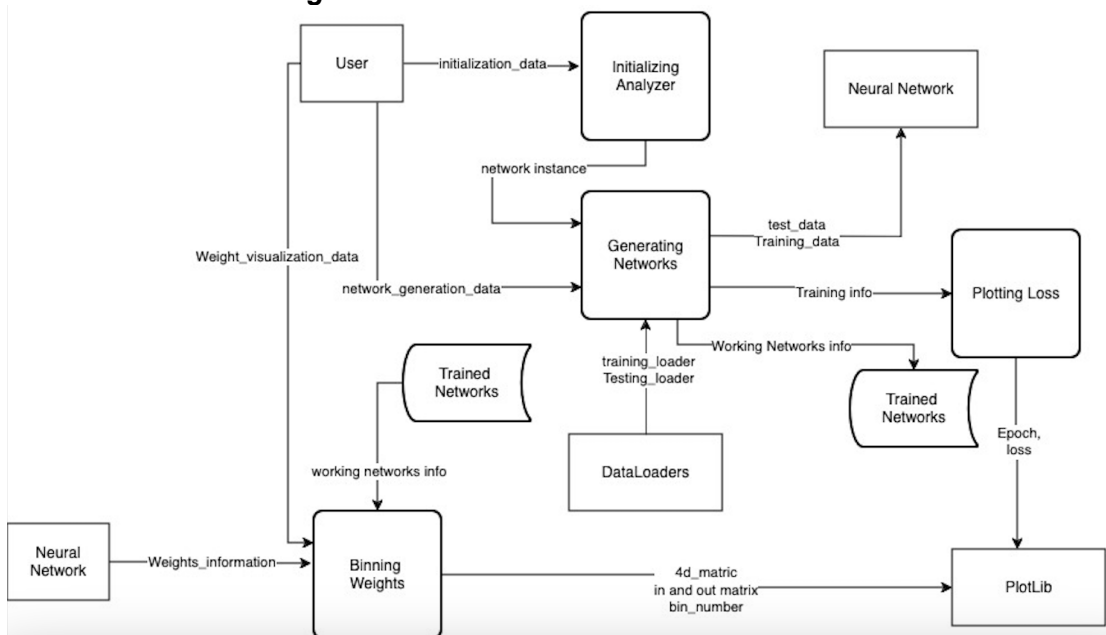
Name	Description	Structure	Range
<i>Training data</i>	<i>The dataset used to train neural networks.</i>	<i>DataLoader</i>	<i>N/A</i>
<i>Number of networks</i>	<i>The total number of neural networks to be trained in one epoch.</i>	<i>int</i>	<i>>=1</i>
<i>Bins</i>	<i>The number of bins used for weight in discretization in histograms</i>	<i>int</i>	<i>30 as default</i>
<i>Epoch data/results</i>	<i>Stores loss per epoch to track convergence.</i>	<i>List[float</i>	<i>N/A</i>
<i>Initialization_data</i>	<i>Contains parameters for network training initialization: neural_network_class,</i>	<i>dict</i>	<i>N/A</i>

	<i>amount_to_produce, success_loss, convergence_threshold, max_attempts.</i>		
<i>Network_generation_data</i>	<i>Contains parameters used during training: num_epochs, loss_fn, learning_rate</i>		

7.1.4. Context Diagram (Level 0 Data Flow Diagram)



7.1.5. Level 1 Data Flow Diagram

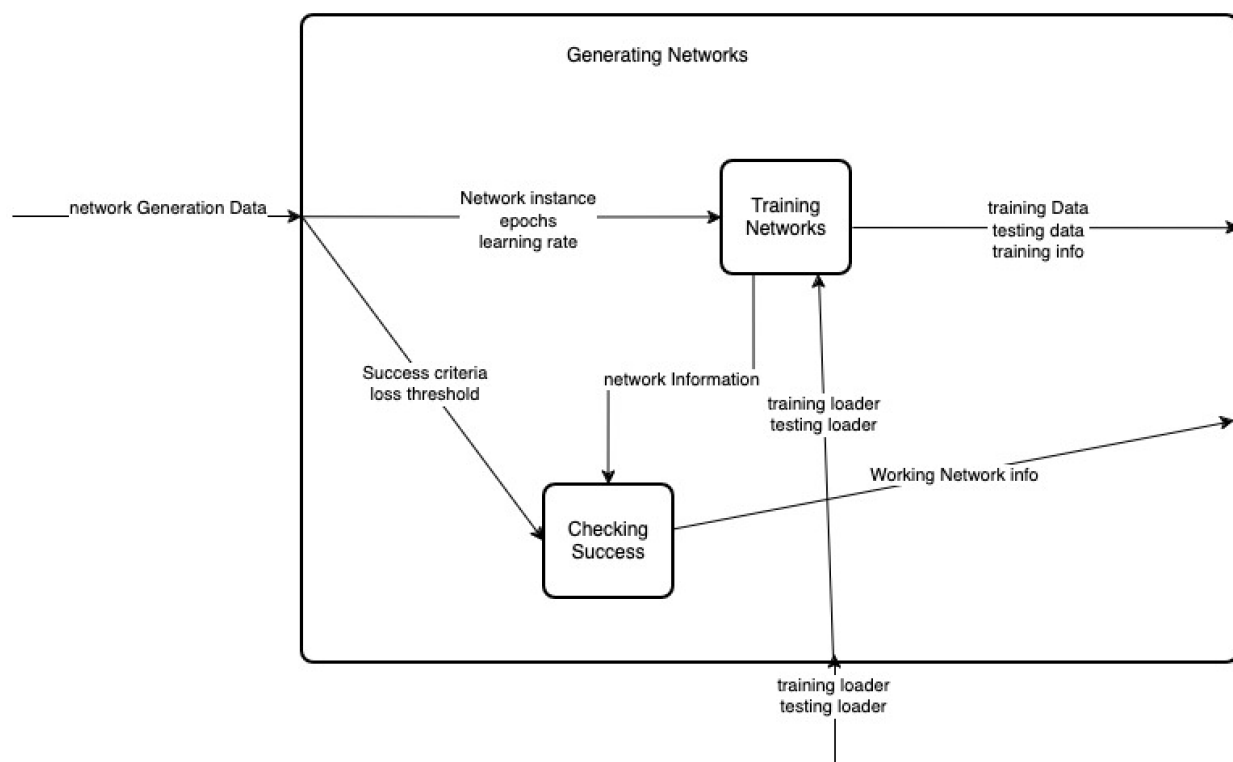


Processes in DFD Level 1

- *User* = provide initialization data to define network generation settings.
- *Initializing Analyzer* = set up training parameters and configure the system.
- *Generating Networks* = create and train multiple neural network using training data and test data from DataLoaders.
- *Trained Networks* = stored networks that have been trained.
- *Plotting Loss* = uses loss values per epoch to send to Plotlib to generate visuals.
- *Binning Weights* = generates a 4D weight matrix using weight values from Trained Networks.
- *Neural Network* = represents the trained models.
- *DataLoaders* = Handles the data from training and testing, which is supplied to Generating Networks.
- *Plotlib* = generates visuals using loss per epoch data from Plotting Loss.

7.1.6. Level 2 Data Flow Diagram

This is the level 2 DFD for the Generating Networks process



Processes in DFD Level 2

- *Training networks = Process which takes in a network, and trains it with the given dataset, and for the given epochs, based on the given success criteria*
- *Checking success = this process simply checks the success criteria and compares the loss with the success criteria. If the success criteria is met, the network is saved.*

Data

- The process Training network takes in:
 - Network instance – a single instance of the network to be trained
 - The number of epochs
 - Learning rate for the optimizer
 - Training loader – the data loader with the training dataset
 - Testing loader – Data loader with the testing set
- And outputs:
 - Training data – information about the training such as loss
 - Testing data – information about testing such as loss
- The process Checking success takes in:
 - Success criteria – loss that the network must meet
 - loss threshold – difference in loss in the last 2 epochs
 - network information – this is the information about the training, which includes the loss and the difference in loss between the last 2 epochs
- And outputs:
 - Working network information – the instance of the working network to be saved for later retrieval

7.2. Class Model

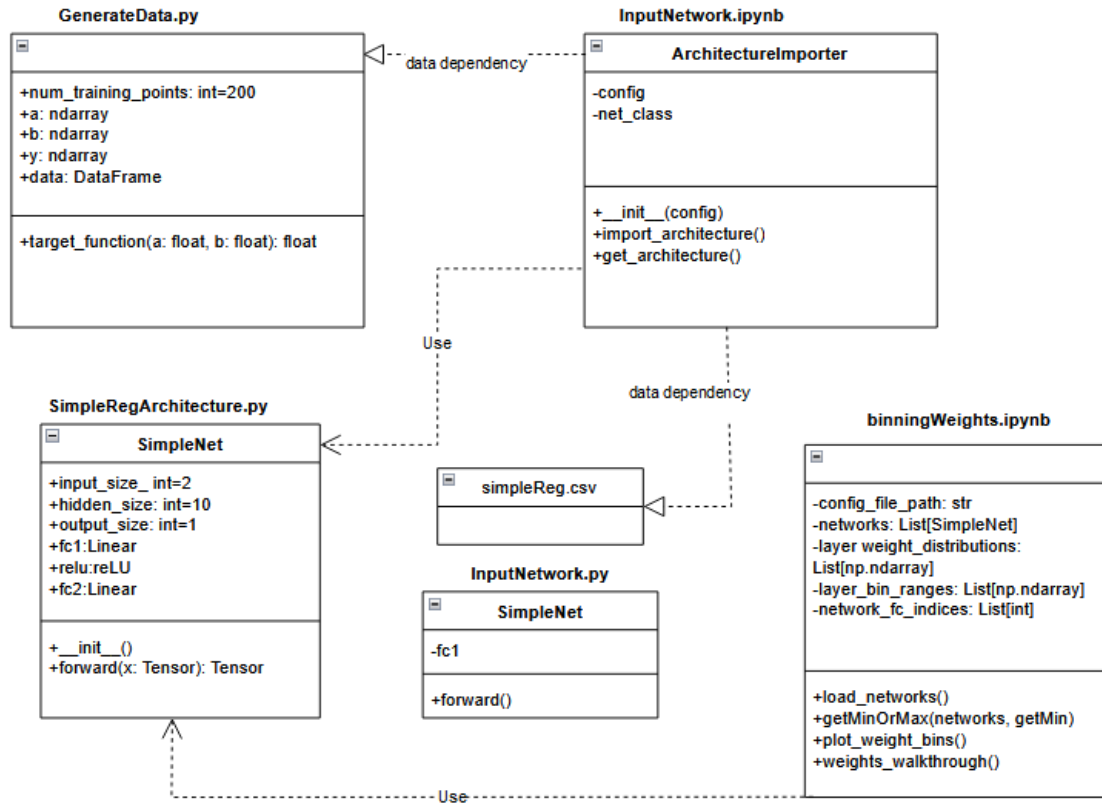


Figure: UML Diagram

7.3. State Model

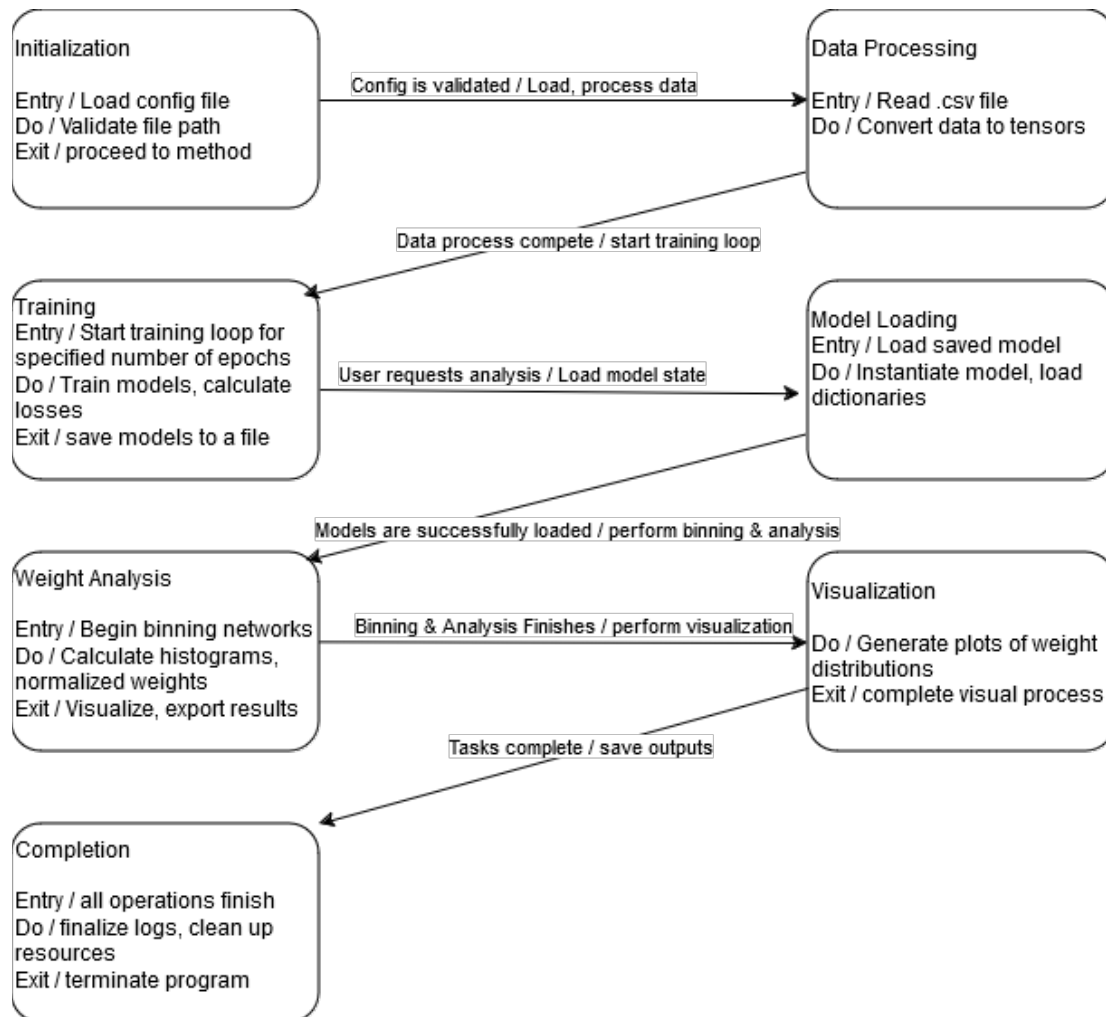


Figure: State Model Diagram

8. Engineering Solutions in Real-World

8.1 Health

Deep learning is already an implemented tool in many health-related fields of modern day society and is a new approach being explored in healthcare. One common usage of deep learning in healthcare is using complex biomedical data in applied models for learning tools [7]. It is suggested that deep learning provides an opportunity for such a tool to be further developed so long as there is improvement in the ease of use of such deep learning models. That is where the project comes into play and assists in not just the ease of use but also the effectiveness of models and model training, which comes with a successful deeper understanding of how the models work. Deep learning not only advances these types of tools for biomedical data but also offers a role of support for clinicians in their lines of work where they might feel overwhelmed [8]. That extra support also means a better, more cared-for experience for patients globally. However, the usage of AI in healthcare specifically is discussed to have a multitude of negative effects, one of them being bias in datasets or biomedical data used. This data may not align with more diverse and sometimes unpredictable the real-world applications it is being trained for. Such as one model may be trained based off data from an urban-established hospital, but the model is applied in a completely different community zone. Another negative part of AI in health to consider is the usage of black-box reasoning within the algorithms of these models[9]. This all means that the decision-making is not completely transparent to humans. Meaning the more complex the problem, the harder it is for a provider to explain why their DL models may have reached a certain diagnosis or conclusion. This can lead to distrust in the AI from a clinical perspective and ultimately a decline in its development of being used in healthcare, as providers may find it to be more of a liability. Outside of healthcare, AI is still used for many health-related jobs for public wellbeing such as fire-detection(OCFA AI Fire-Detection), AI-powered disaster relief drones(food and medical deliveries), and even detecting mass disease before it spreads (BlueDot's detection of COVID-19 before outbreak).

8.2 Safety

This project aims to aid in filling a gap in our understanding of AI and ML. We believe our project has the potential to have some impact in the safety and security aspects of ML. Our project aims to improve our understanding of the weight distribution patterns of neural networks. As of now, neural networks operate as 'black boxes' where the internal logic used to produce their outputs is not understandable. If we are able to understand the inner workings of these neural networks through the analysis of their weight distribution patterns, this could help illuminate the 'black box' of neural networks. Understanding what these weights mean and how they operate is imperative for safety-critical applications. For example, in airplanes, self-driving cars, in cybersecurity applications, data-sensitive applications, and more. This helps reduce the risk of having catastrophic events due to our lack of understanding and optimization of these models.

8.3 Welfare

AI systems built on neural networks seem poised to revolutionize global welfare through a number of vectors. Large Language Models are being used for search, programming, document creation and more; all while robotics are on the precipice of becoming mainstream. The fundamental building block of the intelligence behind these technologies is the neural network and the fundamental building block of neural networks is the weight. In this new technological revolution, it is imperative to our welfare we develop methods to efficiently and effectively analyze the weights of these neural networks. Better understanding of neural network weights leads to better neural networks which in turn leads to welfare increases. Increases in welfare come via more efficient neural networks, but also safer neural networks. Through the use of our network training and analysis tools developed in this project, one can explore the distributions of weight values in networks and could feasibly a) understand how weights are used to store individual pieces of information or b) develop more efficient methods of weight initialization such that networks converge to some distribution faster. Both of these options lead to better AI systems based on neural networks. As to the question of whether stronger AI systems leads to increased welfare for humanity, Dario Amodei, CEO of the AI lab Anthropic put it well in his essay *Machines of Loving Grace*: “I think that most people are underestimating just how radical the upside of AI could be, just as I think most people are underestimating how bad the risks could be”[13]. This sentence encapsulates our situation well, AI has the capacity to transform or destroy our wellbeing so it’s critical we have a fundamental understanding of the weights which comprise these systems.

8.4 Real-World Analysis (only need 4)

- **Global effect** – Daniel Machado

Our project should have different impacts on different parts of the world, due to the transparency and modularity it would allow ML models to have. In some regions where governments might be more “tyrannical” where information is censored, and the outputs of the model have to be monitored, our project would allow engineers to understand what the model is learning, and optimize for those required results (censorship in this case). There is so much that current techniques to mold LLM outputs can do, meaning that even after applying these techniques the LLM might still produce undesired outputs. By using our project to analyze what the LLM is learning, Engineers will be able to shape and optimize the LLM exactly how they desire, rather than having to filter out the outputs if the model produces an unwanted response.

Also, our project might be able to help in different industries. By being able to shape the ML models as desired, we are able to optimize in different aspects such as safety. Making models robust and more trusted to be used in safety-critical applications. Thus, allowing for innovation in areas where ML is not trusted and might have taken a long time to adopt.

Our project should also have a main impact in ML training and cost. By optimizing for a wanted solution, we avoid the ML model learning data that is too broad. We are guiding

towards the solution we want, thus reducing the amount of resources and time needed to train these models. And not only training will be cheaper and less intensive, but also running these models. This will allow ML models to become extremely good at a single task – kind of how a CPU is good for a broad amount of things, while a GPU is extremely good at one thing (Mathematical calculations). This might allow the development of really small, embedded ML models which do not need a powerful gpu or cpu to perform their tasks. Ultimately, this allows ML models to be integrated more into our society, making optimization for different systems easier than ever.

- **Social** – Muhammed-Sharif Adeptu

The social impacts of our project are mainly going to be felt within the Artificial intelligence development community, people who research neural networks and work on these large-scale Artificial intelligence architectures will now save an immense amount of time and resources while they develop these networks. Our project helps those in the research space to efficiently analyze their and spend more time working on developing their work. In *“the PHD Experience: a review of the factors influencing doctoral students’ completion, achievement and well-being”* [12] the authors mention that Gordon (2003) has described common difficulties encountered by doctoral students in the dissertation process, and how “being unprepared for how enjoyment of the scientific method can often turn to frustration in the process of conducting real-world research” our goal social impact is to mitigate this frustration at the scientific method, hopefully increasing the number of Artificial intelligence researchers who finish their programs.

Additionally, in the same paper, we are given the impact of research on PHD candidates social lives and vice-versa. This reported lack of leisure time has been found to correlate with higher levels of burnout, leading many Researchers to ultimately either give up on their projects or gain increased levels of depression (Uqdah, Tyler, & DeLoach, 2009) and impairing short term and long-term relationships with friends. (Wellington and Sikes (2007)). Much of this burnout stems from working long hours, combined with progressing too slowly. Our tools aim to help increase the speed at which neural network analysis is done. A main societal benefit of our project will be a decrease in PHD burnout and frustration. Ultimately shifting the work-life balance of these researchers towards something more positive.

- **Environmental** – Simon Cole

AI systems fundamentally based upon neural networks have already become an integral part of our society and seem poised to revolutionize how we work and how we live in the next decade. The key hypothesis driving the current boom in AI has been the proposition that a network trained with more computational resources or more data will increase in intelligence. This hypothesis was first formalized in Jan 2020 by Kaplan et al. and became known as “The Scaling Laws” [5]. These “laws” have held up remarkably well since their establishment and continue to be pushed by newer and larger large language models to this day. The

establishment of these laws is critical for the future of the environment because larger large language models means more energy required to train them.

Kasper Ludvigsen in his article published in Towards Data Science [6] estimates OpenAI's GPT-3 model emitted 552.1 tons of CO₂e emissions in training alone. This staggering number surely rose even higher for OpenAI's subsequent models and will likely only grow in the future. As climate change continues to become an ever more existential threat for our society, it is critical we develop more energy efficient methods for training these neural networks. This is where our project comes into play from an environmental standpoint. Our rigorous analyses of weight distribution patterns in neural networks will lead to a greater understanding of how neural networks learn and how to get them to learn more efficiently. Specifically, better methods of initializing neural networks before training are poised to be developed which could lead to shorter training times and thus fewer carbon emissions.

- **Economic** – Leah Powell

The usage of AI and deep learning in economics has so far proven to be beneficial in many ways that are described by Artur Haponik of *.Addepto*, a company that specializes in AI-based consulting approaches[10]. These benefits include but are not limited to improvement of productivity, product enhancement, and stimulating new companies. Many firms that have begun studying and implementing the use of AI in their financial consulting produce results like the ones mentioned before. Our project would leverage those already well-performing AI tools to be even more of an assistance to marketing companies all around the world. In discussion of employment, AI has been a hot topic for decades, with most people concerned about the negative side affects on employment. Our project would offer AI to further stabilize its role in the job market and would pose as a competitor in employment for non-complex and non-creative job roles.

However, there are efforts being made to thwart this negative effect and expand upon AI's role in the *creation* of more jobs, one of them being Sustainable Development Goal 8 [11]. This is a project which aims to promote not just fuller employment but ensure productive employment practices by analyzing the job market in China between 2006 and 2020 in relation to AI 'stealing jobs'. The study found that more often than not, AI actually introduces more jobs, as workers using AI are more productive and more investment in new technologies due to more productivity leads to more job opportunities. Their study even goes into discussions on government role in job management and overlooking in light of AI, suggesting there should be a system of channels for not just monitoring how AI is taking away jobs from humans but also using AI to be transparent about the current job market. This would include job forecasting and anticipating structural unemployment changes [8]. It should be noted that this project, being implemented in data from China, is still a work in progress and would need more global data for global implementation. Our project could prove to be a contributor the its globalization of this approach, in improving the ease-of-use for this models and deepening our understanding of how these forecasts were found by algorithms, as opposed to concrete black-box reasoning.